

Your Team Got Faster. It's Still Not Shipping.

Let me guess what's happening on your team. PR count is up, way up. Your engineers tell you they feel two or three times faster with AI, and the dashboards back it. And yet your on-call is drowning, the incident tracker is worse than last quarter, and every retro ends the same way: "we're productive, so why is nothing stable?"

You've started to suspect velocity is a decoy. You're right. Here's what you should manage instead.

The number everyone got wrong

The cleanest read on this comes from Google's DORA research, the industry's most-trusted survey program and the one everyone quotes on delivery performance. In its 2024 Accelerate report, DORA found that a 25% increase in AI adoption was associated with a 1.5% drop in delivery throughput and a 7.2% drop in delivery stability.

Read that twice. Individual productivity is up: more than a third of respondents reported moderate-to-extreme gains, and over three-quarters now rely on AI for at least one daily task. Meanwhile delivery, the thing you actually get paid to produce, went down. DORA put it plainly: "improving the development process does not automatically improve software delivery — at least not without proper adherence to the basics of successful software delivery, like small batch sizes and robust testing mechanisms."

Individual speed and team effectiveness have decoupled. That's the finding, and it's not an accident of the data. It's the mechanism.

The vendor who sharpened it, and picked a fight

Now bring in Faros AI. They're a vendor of engineering-analytics software, so treat their numbers as telemetry with a commercial angle — but it's the most granular telemetry in the space: two years of data across 22,000 developers and more than 4,000 teams. Their "Acceleration Whiplash" study says individual output exploded, tasks per developer up 33.7% and epics per developer up 66.2%. And the cost is downstream: bugs per developer up 54%, incidents per pull request up 242.7% (more than tripled), median time in review up 441.5%, unreviewed merges up 31.3%.

So both sources agree on the big picture: output exploded, stability didn't. But they disagree on something important. DORA's 2025 AI report argues that mature teams, the ones with strong foundations and good practices, are protected, that the basics amplify AI's benefits. Faros says outright: "high performers aren't insulated. Engineering maturity is not a shield."

Which is right? Genuinely open question. DORA runs self-reported surveys; Faros runs before/after telemetry on its own customers. Don't let the disagreement stall you. The fact that the two most credible sources can't settle it tells you the bottleneck isn't the technology. It's the system underneath, the part nobody's measuring. That's exactly where your lever is.

Three mechanics, three levers

Review is now the bottleneck. Amdahl's law: a system moves only as fast as its slowest link. AI raised the write-speed of every engineer, but it didn't raise the human review system's capacity to absorb the output. So the queue balloons, time-in-review climbs 441%, and your reviewers, buried, either skip review entirely (that 31% unreviewed-merge rate) or skim the surface and let the defect sail through to

production, where it shows up as your on-call. The slowest link in your pipeline is now a human reading someone else's AI-generated diff. Lever: a review rule. Define what "done reviewing" actually means, and cap how much one person is expected to absorb in a day. Absorb capacity is the metric to watch, not velocity.

Batch size broke. Small batches and robust testing are literally the "basics" DORA names as the price of process-speed gains. But AI makes it trivially easy to generate a 600-line diff, and every review and every defect surface scales with it. Bigger batches spike cognitive load on the reviewer at the exact moment volume spikes, so quality drops as volume rises. Lever: a batch cap. Set a size limit on what merges unreviewed and what needs a split before review. This is the highest-leverage, lowest-resistance change you'll make.

Ownership got taxed. Here's the painful one. AI code is stylistically fluent but structurally untrustworthy, the failures under the surface. A correct review means reconstructing the author's intent, which is slow, expensive cognitive work. That work lands on your deepest-knowledge engineers. Call it the senior engineer tax: the people who know the system best absorb everyone's review and rework, so their own delivery collapses while the rest of the team feels unstoppable. Lever: a named owner per pipeline slice. Code, review, integration, release. Each stage gets one person accountable for flow, not just completion. Distribution of load is a system decision, and right now you're letting it happen by accident.

The five-step pipeline audit

Run this on one team this week. It takes an afternoon and it will tell you exactly where your system is breaking.

1. Size it. Pull the last 30 days of pull requests. Flag anything over roughly 400 lines. That's a worked rule of thumb, not a law, but it's where review load and defect surface spike. Bigger than that and your reviewers are running on fumes.
2. Count the unreviewed. What percentage merged with no human or agentic review at all? That number is your silent failure tap. If it's climbing, that's your on-call next quarter.
3. Find the reviewer choke point. Who holds the longest unreviewed queue and the longest time-in-review? That's your slowest link, your Amdahl's-law bottleneck, personified.
4. Name an owner per slice. Code, review, integration, release. One named person accountable for flow through each stage, not just for their own completion.
5. Pick one lever. Change exactly one thing this sprint: a batch cap, a review rule, or a handoff owner. Measure the same metric next sprint. Then move the next lever.

What it looks like when it works

Before: your team ships 40 PRs a sprint, feels fast, and three of them explode in production every two weeks. Unreviewed merges are north of a quarter, one senior engineer's name is on half the review queue, and nobody can say who owns release flow.

After one sprint of this audit: you cap batch size so nothing big merges without a split, the reviewer queue visibly shrinks, and the senior engineer isn't the one absorbing everything. Six weeks in, your PR count is roughly flat, but your incidents and your unreviewed merges are down, and for the first time in months "productivity" and "stability" point the same direction.

That's the whole game. Stop managing the pace; it was never the constraint. The constraint is the system that turns output into shipped, stable software, and it's the one thing you're actually in charge of. Audit it, name an owner, pick a lever, measure it. The team that gets faster and stays stable is the one that treats the pipeline as the product.